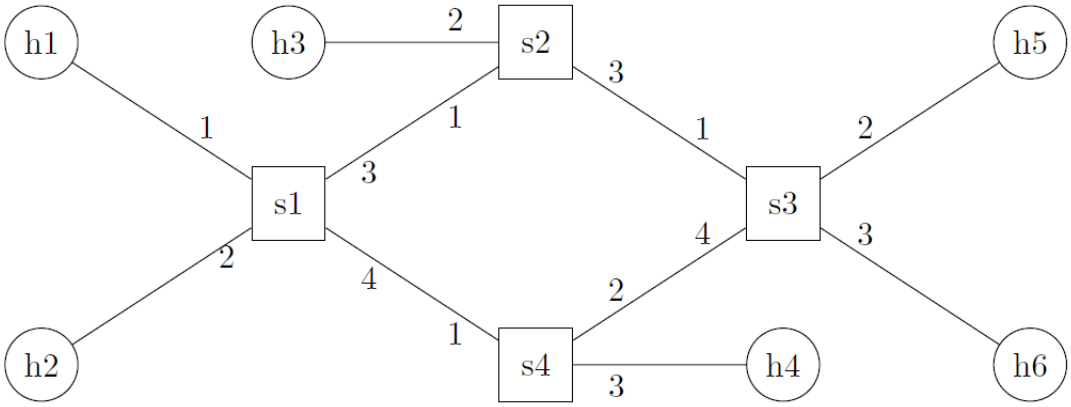
Software Defined Networking

(simulazione del 03/06/2020)

Esercizio 1

Si consideri la rete in figura. I nodi h1,. . . , hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,. . . , hn sono MAC1,. . . , MACn e IP1,. . . , IPn. Sui nodi h1,. . . , hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Se non altrimenti specificato, il TTL iniziale è 200.



Negli switch sono preinstallate le seguenti regole.

Switch 1

Priority	Match	Action
5	*	Output(FLOOD)
10	eth_dst = MAC2	Output(2)
10	eth_dst = MAC3	Output(CONTROLLER,128)

Switch 2

Priority	Match	Action
0	*	Output(CONTROLLER,128)
10	eth_type = IPv4 ip_proto = TCP	DecTTL Output(FLOOD)

Switch 3

Priority	Match	Action
5	*	Output(FLOOD)
10	eth_dst = MAC5	Output(2)
10	eth_dst = MAC3	Output(CONTROLLER,128)

Switch 4

Priority	Match	Action
0	*	Output(CONTROLLER,128)
10	eth_type = IPv4 ip_proto = UDP	Output(FLOOD)

Il controllore Ryu è collegato ai nodi s1,. . . , sm ed è configurato come indicato nel seguente programma. Le interfacce tra gli switch e il controllore non sono indicate in figura.

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
    parser = datapath.ofproto_parser; in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
    buffer_id = msg.buffer_id; data = msg.data

    if (datapath.id == 1 and pkt_tcp is not None):
        match = parser.OFPMatch(eth_dst=pkt_eth.dst, eth_type=ETH_TYPE_IP,
            ipv4_dst = pkt_ipv4.dst, ip_proto=IPPROTO_TCP,
            tcp_dst=pkt_tcp.dst_port)
        actions = []
        if(pkt_tcp.dst_port<1024):
            actions.append(
                parser.OFPSetField(tcp_dst = pkt_tcp.dst_port + 1024 )
            )
        actions.append( parser.OFPActionOutput(FLOOD) )
        inst = [parser.OFPInstructionActions(
            ofproto.OFPIT_APPLY_ACTIONS,actions
        )]
        mod = parser.OFPFlowMod(datapath=datapath, priority=20,
            match=match, instructions=inst, buffer_id=buffer_id)
        datapath.send_msg(mod)
```

L'host h2 porta 1234 apre una connessione TCP verso h5 porta 5678.

1. Quanti pacchetti escono dall'interfaccia 2 dello switch s2?
2. Quanti pacchetti escono dall'interfaccia 1 dello switch s2?
3. Scrivere il MAC destinazione del pacchetto uscente dall'interfaccia 3 dello switch s3.
4. Scrivere il TTL del pacchetto uscente dall'interfaccia 3 dello switch s3.
5. Quanti pacchetti arrivano in h5?
6. Scrivere il TTL del pacchetto che arriva in h5.

L'host h5 risponde

7. Quanti pacchetti arrivano in h2?
8. Scrivere il TTL del pacchetto che arriva in h2.

L'host h2 porta 3322 apre una connessione TCP verso h3 porta 123

9. Scrivere il numero di porta sorgente del pacchetto che arriva in h3
10. Scrivere il numero di porta destinazione del pacchetto che arriva in h3
11. Scrivere che tipo di pacchetto TCP viene generato da h3 in risposta
12. Scrivere il numero di porta sorgente di questo pacchetto quando arriva in h2
13. Scrivere che tipo di pacchetto TCP viene generato da h2 in risposta
14. Scrivere il numero di porta destinazione di questo pacchetto quando arriva in h3

Esercizio 2

Un filtro di Bloom a conteggio è costituito da un array *A* di *m* numeri interi e *k* funzioni hash. Nel filtro a conteggio ogni volta che viene inserito un elemento x, le celle di *A* in posizione $h_i(x)$ $\text{con } 1 \leq i \leq k$, sono incrementate di 1. Nel filtro a conteggio è possibile rimuovere un elemento decrementando le celle corrispondenti.

Un sistema di filtraggio delle connessioni TCP usa un filtro di Bloom a conteggio. Nel momento in cui una connessione TCP è autorizzata i suoi parametri sono inseriti nel filtro.

Al termine della connessione gli stessi parametri sono rimossi. Si supponga che il filtro contenga al massimo 3 000 connessioni.

Il filtro è costituito da un array di m = 2^12 celle e k = 3 funzioni hash.

15. In un dato momento, nel filtro sono presenti 1000 connessioni. Qual è la probabilità che una nuova connessione non autorizzata sia ammessa nel sistema? [troncare alle prime due cifre significative]
16. Calcolare la grandezza minima del filtro (in bit) per non avere mai overflow.
17. Calcolare il numero minimo di celle dell'array per avere una probabilità di falso positivo sempre inferiore a 10^−3.
18. Si consideri la famiglia di funzioni hash $h_i(x) = 2i + 4x \bmod m$. Inserire le connessioni con gli identificativi 2 e 3. Mostrare il valore della cella con indice 14 dopo gli inserimenti.